

Broadcast Channel POC

Synchronised playback across many TVs, using a linear HLS channel instead of client-side coordination.

Component	Technology
TV application	LightningJS (Blits 2.10) + hls.js, packaged with Capacitor
Channel server	Node.js, zero dependencies
Encoder	ffmpeg — one process per channel
Transport	HLS with EXT-X-PROGRAM-DATE-TIME
Targets	Android TV today; webOS and Tizen share the same build

What problem this solves

Several televisions must show the same content at the same moment. A new TV switched on at any time must join whatever is already playing, not start from the beginning. Nobody should have to configure a screen after the first setup.

Contents

- 1 Overview and the two approaches
- 2 System architecture
- 3 How an MP4 becomes a channel
- 4 Keeping televisions in sync
- 5 Pairing a television
- 6 Screen wireframes
- 7 API reference
- 8 Running the system
- 9 Known limitations

SECTION 1

Overview and the two approaches

There are two ways to make several televisions show the same thing, and they differ in **where the intelligence lives**.

	Client-side sync	Broadcast channel (this project)
Idea	Each TV computes where it should be and forces itself there	The server produces one live stream; TVs simply play it
TV code	~1,700 lines of schedule and position arithmetic	~20 lines: open a URL and play
Clip transitions	Every TV reloads its player — a visible black gap	Invisible; just another segment
Content change	A version protocol must reach every TV	Server-side only; TVs untouched
Server cost	Almost none	One encoder process per channel
If the server dies	TVs keep playing from cache	Everything stops

Why broadcast wins here

Client-side sync asks **N unreliable clients to independently agree**. Correctness depends on every TV reaching the same answer, continuously, using its own clock, its own network and its own buffer state.

Broadcast asks **one controlled server** to produce a stream. There is no agreement to maintain because there is nothing to disagree about. The expensive problem is deleted rather than solved.

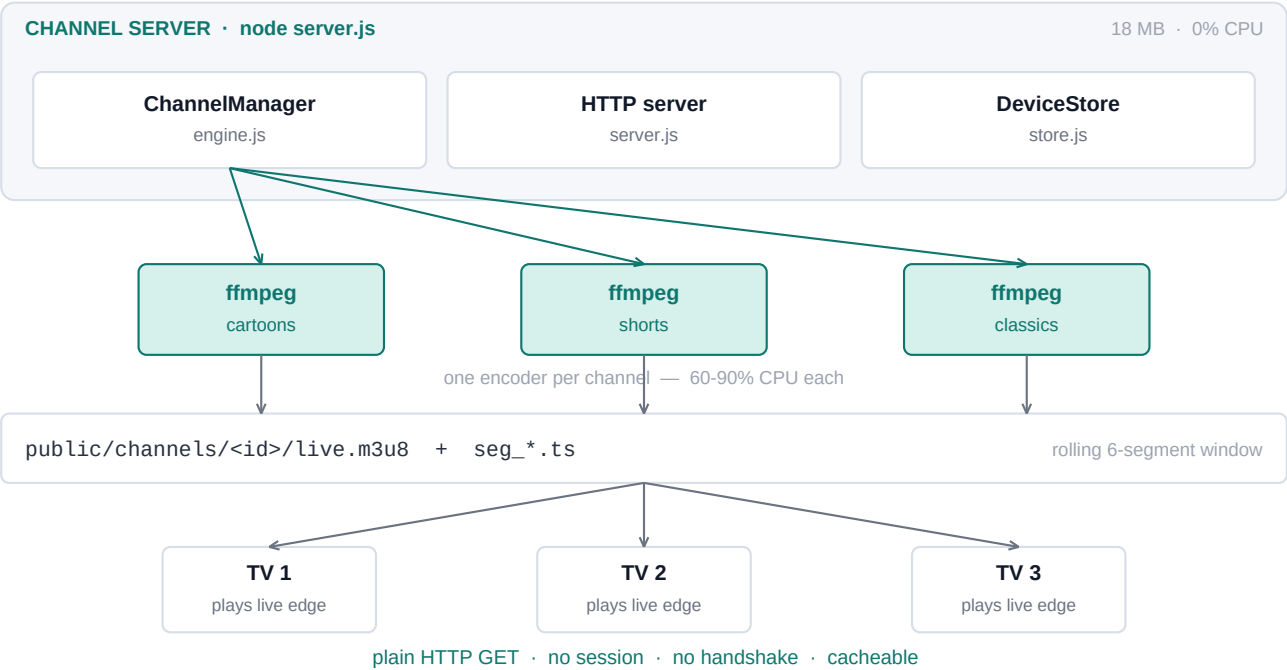
The honest trade-off

Broadcast centralises risk. Client-side sync degrades gracefully — if its server dies, TVs keep playing from cached data. If the channel engine stops producing segments, every screen stops together. That makes supervision and monitoring mandatory rather than optional.

SECTION 2

System architecture

One Node process supervises everything and serves files. It never touches video data — that is entirely ffmpeg's job.



The measurement that explains the design

On a running system, Node uses **0% CPU and 18 MB** of memory. The three ffmpeg processes use **60–90% of a core each**. Node starts processes, watches them, and pipes files off disk; everything expensive happens inside ffmpeg.

Adding...	Cost
1,000 more televisions	Almost nothing. Static file reads, and a CDN absorbs them entirely.
One more channel	A whole additional encoder — 60–90% of a CPU core.

Cost scales with **channels, not viewers**. This is the opposite of the intuition most people bring to video systems, and it is the single most useful thing to know when planning capacity.

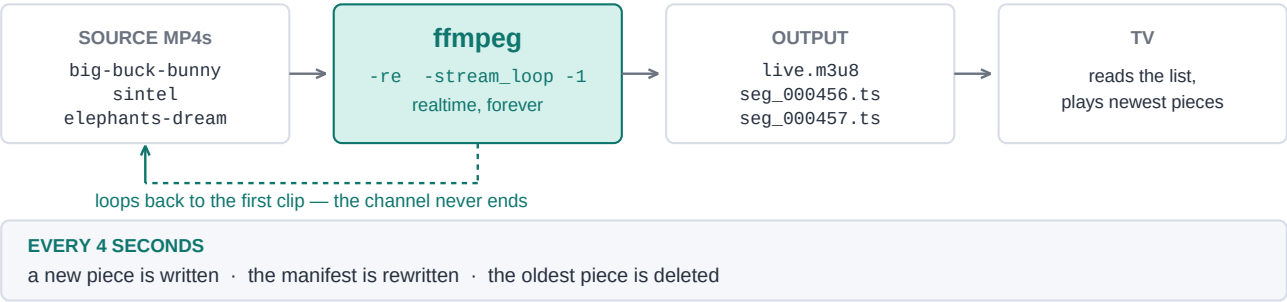
Separation of concerns

The video path depends on neither the device store nor the API layer. If both were deleted, every already-paired television would keep playing indefinitely, because playback is nothing more than repeated GETs of static files. Losing the control plane costs you the dashboard, not the picture.

SECTION 3

How an MP4 becomes a channel

A television cannot join the middle of a file. So ffmpeg continuously chops the source video into small pieces and maintains a short list of the newest ones.



This is not ordinary conversion

Normal conversion runs once and produces a complete set of files. This runs **at realtime speed, forever**, and keeps only the newest handful of pieces.

Observed on a running channel	
Time on air	30 minutes
Pieces created	457 (seg_000000 -> seg_000456)
Pieces still on disk	7
Source MP4s	never modified — read again and again

The command

```
ffmpeg -re -stream_loop -1 -f concat -safe 0 -i playlist.txt \
-c:v libx264 -g 50 -c:a aac \
-f hls -hls_time 4 -hls_list_size 6 \
-hls_flags delete_segments+append_list+program_date_time \
public/channels/cartoons/live.m3u8
```

Flag	Why it matters
-re	Paces reading at realtime. Without it ffmpeg races through the files and nothing is live.
-stream_loop -1	Loops the playlist forever — the channel never ends.
-f concat	Treats the playlist as one continuous input, which is why clip transitions are invisible.
-hls_time 4	Four-second pieces; -g 50 places a keyframe every two seconds so pieces always split cleanly.

delete_segments	Stops the disk filling over days of uptime.
program_date_time	Stamps every piece with a wall-clock time. This is what makes frame-accurate sync possible.

Why re-encode instead of -c copy

Copying is far cheaper and is the right choice once all source files share identical parameters. But looping a concatenated playlist with -c copy produces timestamp discontinuities at every wrap, which some players show as a stutter. Re-encoding regenerates clean timestamps.

SECTION 4

Keeping televisions in sync

Every TV pulling the same manifest guarantees they show the same **content**. It does not put them on the same **frame**. Each player picks its own start point near the live edge, and once two TVs drift apart nothing pulls them back — live HLS has no restoring force.

So each TV holds a fixed latency behind a shared clock:

```
target = serverNow() - 12000ms
drift = hls.playingDate - target           // playingDate comes from PDT
```

|drift| > 5000 ms

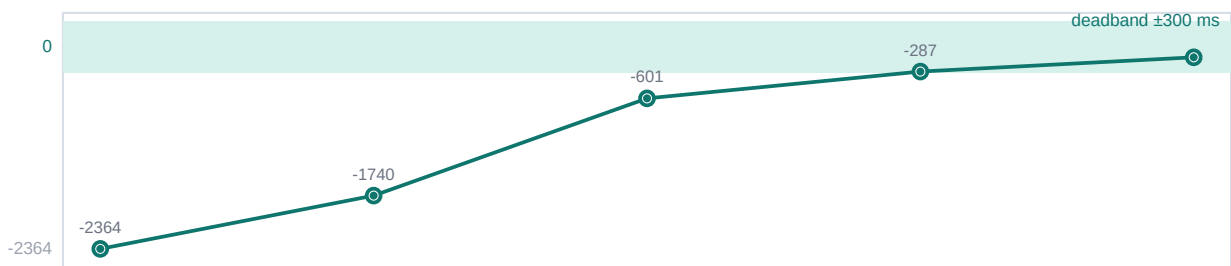
seek

300 - 5000 ms

playbackRate 0.98 / 1.02

<= 300 ms

leave it alone



measured on an Android TV emulator: drift closes at exactly the 2% that 1.02x predicts

The server clock

serverNow() is round-trip corrected: t0 is recorded **before** the request and the offset is taken from the midpoint, sampled several times with the lowest-latency result kept. Measuring after the response arrives bakes in half the round trip and biases every television differently.

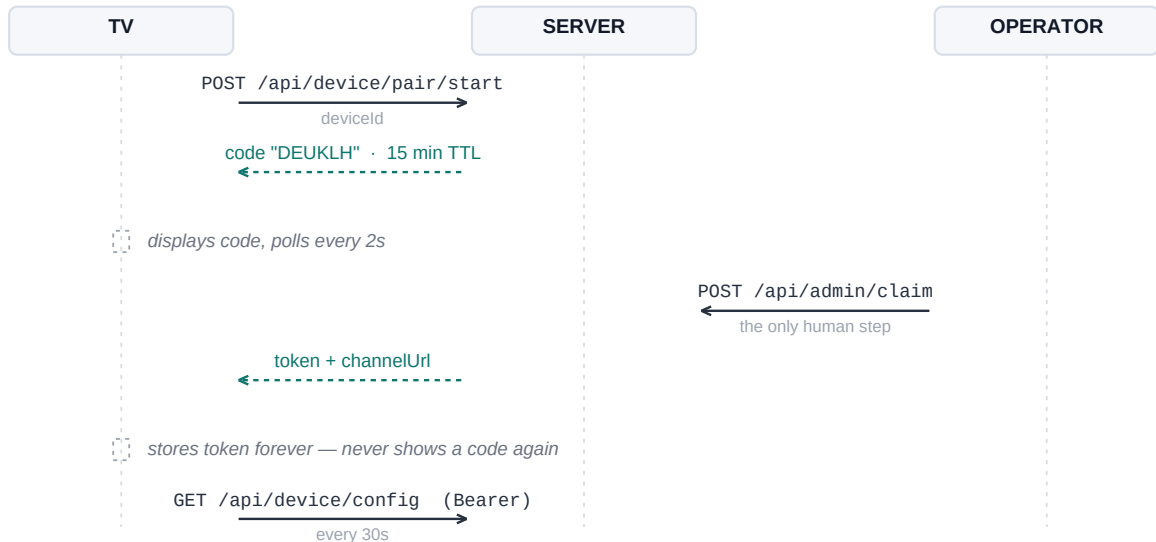
Why this matters more than it sounds

During testing an Android emulator had a clock 10.5 minutes out from the host. Without correction that TV would have shown content ten minutes away from every other screen. The correction absorbed it silently, and the two clients still agreed on frame.

SECTION 5

Pairing a television

A television is an appliance, not a personal device. It is authorised once by an operator and never asks anyone for anything again.



The code is a doorbell, not a key

The six-character code grants nothing. It exists only so a human can point at one specific television among many. The real credential is the token, which the server creates **after** approval.

Property	Value	Reason
Alphabet	A–Z and 2–9, minus I, O, 0, 1	Read off a screen across a room and typed elsewhere
Length	6 characters	$32^6 \sim 1.07$ billion combinations
Lifetime	15 minutes	Long enough for setup, short enough to expire
Generated with	crypto.randomInt	Math.random is not suitable for credentials
Token	48 hex characters	Issued only on approval; stored permanently

Why not a login screen

Consider a building-wide power cut at 6am. With a login screen, somebody walks to every television and types an email address using a remote control on an on-screen keyboard. With pairing, every screen returns on its own. This single consideration drives the entire design.

Unpairing

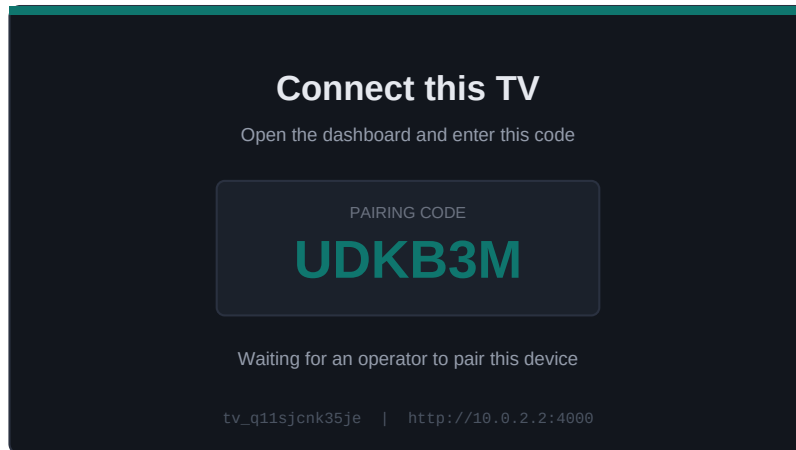
Removing a device deletes its record and token. The television is not notified — nothing is ever pushed to a television. Within 30 seconds its heartbeat returns 401, at which point it clears its stored token and displays a fresh pairing code.

SECTION 6

Screen wireframes

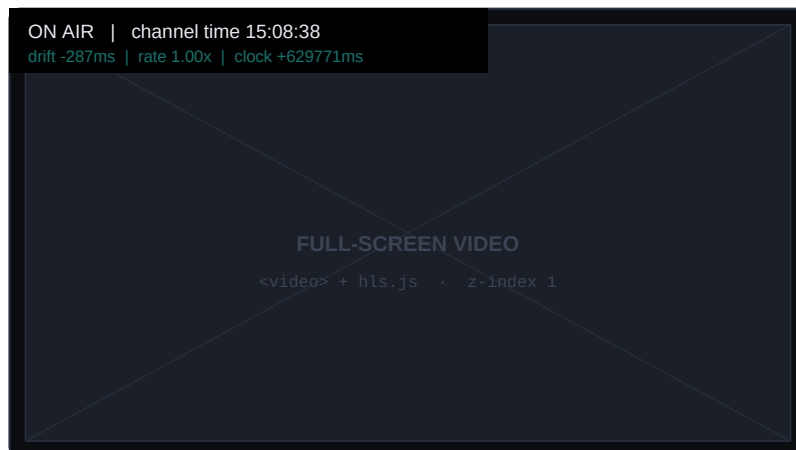
Three screens exist in the entire system. Most televisions only ever display the second one.

6.1 Television — pairing (first boot only)



Shown once per TV, ever. After pairing this screen is never seen again — not on reboot, not after a power cut, not after an app update.

6.2 Television — player (every boot thereafter)



Lightning canvas is layered ABOVE the video (z-index 2) so overlays draw on top. It must stay transparent — an opaque root hides the picture while text still shows.

6.3 Operator dashboard (browser)

Channel Control

Each channel is its own continuously-encoded HLS stream.

CHANNELS

● Cartoons	3 videos	live · 0 restarts
● Animation Shorts	3 videos	live · 0 restarts
● Classics	3 videos	live · 0 restarts

PAIR A TV

Pair device

DEVICES

NAME	STATUS	PLAYING	
TV 1	● online	Cartoons v	Remove
TV 2	● online	Classics v	Remove
TV 3	● waiting · A3CK47	— v	Remove

The dashboard is the only place a television can be authorised, moved between channels, or removed. Channel health, uptime and restart counts are shown per channel, and each paired television carries a dropdown that reassigns it to a different playlist. The change reaches the TV on its next heartbeat, within 30 seconds.

SECTION 7

API reference

Every endpoint lives in a single file, `channel-server/server.js`, matched by a plain "METHOD /path" string. There is no framework and no router library.

Device endpoints

Endpoint	Purpose
POST /api/device/pair/start	TV requests a pairing code. Unauthenticated.
GET /api/device/pair/status	TV polls every 2s until an operator claims it.
GET /api/device/config	Boot config and 30s heartbeat. Bearer token. The only authenticated call.

Operator endpoints

Endpoint	Purpose
POST /api/admin/claim	Authorise a device onto a channel. The system's trust boundary.
POST /api/admin/unpair	Delete a device and its token.
POST /api/admin/device/channel	Move a television to another channel.
GET /api/admin/devices	Fleet list with online state.
GET /api/admin/channels	Channel list with health.
POST /api/admin/reload	Restart one encoder after a playlist change.
GET /api/status	All channel health plus server time.

The video path

Endpoint	Purpose
GET /channels/<id>/live.m3u8	The channel. No token, no session, no headers.
GET /channels/<id>/seg_*.ts	Media segments. Immutable once written.

Cache headers decide whether this works

Manifests must be sent no-store; segments immutable with a long max-age. A CDN caching the manifest stalls every television simultaneously and looks exactly like an encoder failure while you debug it.

SECTION 8

Running the system

Prepare content, once

```
cd channel-server && ./fetch-content.sh
```

Downloads 60-second excerpts from three Blender open movies and normalises them to identical encoding parameters. Every clip in a channel must match exactly, because the concat demuxer joins them without renegotiating.

Start the channels

```
cd channel-server && node server.js
```

Starts one ffmpeg per channel and serves the dashboard on port 4000. No npm install is required — the server has zero dependencies.

Build and install the television app

```
cd tv-app && npm install
npm run android:apk
adb install -r android/app/build/outputs/apk/debug/app-debug.apk
```

Project layout

Path	Contents
channel-server/server.js	All APIs, static serving, cache policy
channel-server/engine.js	ffmpeg supervision, one engine per channel
channel-server/store.js	Device records: codes, tokens, last-seen
channel-server/channels.js on	Channel definitions
channel-server/public/admin.html	Operator dashboard
tv-app/src/pages/Pairing.js	First-boot pairing screen
tv-app/src/pages/Player.js	The entire client: player and latency hold
tv-app/src/utils/device.js	Identity, credentials, server clock

Porting to webOS and Tizen

The television app builds to three plain web files. Nothing platform-specific lives in the player — it is a video element, hls.js and a drift loop. The Android-only pieces are the Capacitor wrapper, MainActivity, and one branch inside `getApiBase()`. Adding a platform means adding a packaging directory and a branch in that one function.

SECTION 9

Known limitations

This is a proof of concept. The architecture is sound; the following must be addressed before production use.

#	Limitation	Impact
1	Operator APIs have no authentication	Anyone who can reach the port can remove televisions or reassign channels. Fix this first.
2	Playlist changes restart ffmpeg	The concat demuxer reads its list once, so changing content is a visible cut on air for that channel.
3	Device registry is a JSON file	Cannot be shared between instances: no horizontal scaling, and a deploy drops pairing state.
4	One origin, no CDN	Every television pulls segments from a single Node process.
5	No alerting	The supervisor recovers a wedged encoder silently. If it failed to restart, nobody would be paged.
6	Audio is muted in the player	A television has no user gesture to unlock audio. The Android WebView flag is already set, so removing one line should suffice.

Verified behaviour

The following were observed on a running system rather than assumed.

Behaviour	Evidence
Drift correction converges	Drift closed from -2364 ms to within the ± 300 ms deadband, at exactly the 2% that 1.02x predicts.
Clock skew is absorbed	A client whose device clock was 10.5 minutes out still agreed on frame with a correctly-set one.
Two platforms stay together	An Android TV and a browser client showed the same frame with no communication between them.
Wedged encoders are caught	An ffmpeg process stopped producing output while still alive; the watchdog detected the stalled manifest and restarted it.
Late joiners land correctly	A television started mid-stream joined the live edge rather than the beginning.

Source: github.com/shashikant8383/VideoSync